Beyond Mode Detection: Reconstructing Detailed Transit Itineraries from Crowdsourced GPS Trajectories

Ojas Jagtap
ojagtap@umd.edu
University of Maryland
USA

Naman Awasthi
nawasthi@umd.edu
University of Maryland
USA

Mohit Jain
jainm27@umd.edu
University of Maryland
USA

Saad Mohammad Abrar
sabrar@umd.edu
University of Maryland
USA

Vanessa Frías-Martínez
vfrias@umd.edu
University of Maryland
USA

Abstract

Reconstructing detailed public transit itineraries from GPS trajectories is crucial for urban transportation planning, yet remains challenging in dense transit networks due to spatiotemporal complexities. Current approaches primarily focus on broad transportation mode detection (walk/bus/rail), failing to identify specific transit lines or stop-level behaviors, which is a critical limitation for operational analytics and equity assessment. This paper presents two novel algorithms to address this gap: a GTFS-based method employing an R-tree spatial index to precisely match trajectory segments with transit routes, and a Google Maps Directions API-based approach leveraging iterative leg matching for route inference. Both algorithms share a unified preprocessing pipeline for trajectory cleaning and staypoint detection, enabling identification of high-resolution transit patterns.

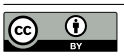
When evaluated on crowdsourced GPS data from Baltimore City's transit system, our GTFS-based algorithm achieved ~80% accuracy in identifying specific transit lines, while the API-based method reached ~85%, significantly outperforming conventional mode detection approaches. These methods also extract new metrics such as actual transfer wait times, boarding/alighting patterns, and route-specific travel durations, which enable targeted service optimizations and deeper transit equity analyses beyond theoretical models. By converting raw GPS data into actionable insights, these algorithms provide urban planners with critical tools to optimize transit service in complex multi-modal networks.

CCS Concepts

• **Information systems** → **Geographic information systems**; **Location based services**; • **Computing methodologies** → **Model verification and validation**.

Keywords

Transit Itinerary Reconstruction, GTFS Spatial Indexing, Multi-Modal Trajectory Analysis, Urban Mobility Equity



This work is licensed under a Creative Commons Attribution 4.0 International License.
SpatialConnect '25, Minneapolis, MN, USA
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2187-8/2025/11
https://doi.org/10.1145/3764924.3770890

ACM Reference Format:
Ojas Jagtap, Naman Awasthi, Mohit Jain, Saad Mohammad Abrar, and Vanessa Frías-Martínez. 2025. Beyond Mode Detection: Reconstructing Detailed Transit Itineraries from Crowdsourced GPS Trajectories. In *The 1st ACM SIGSPATIAL International Workshop on Spatial Intelligence for Smart and Connected Communities (SpatialConnect '25), November 3–6, 2025, Minneapolis, MN, USA.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3764924.3770890

1 Introduction

Understanding individual mobility patterns is crucial for optimizing public transit operations and implementing equitable urban policies. Although GPS-enabled devices have significantly improved data collection resolution, converting raw crowdsourced GPS trajectories into meaningful transit diaries remains challenging, especially when reconstructing detailed transit journeys in dense networks. Analyses primarily focus on broad transportation mode detection (walk, cycling, or generic public transit), but even the latest methods fail to identify specific transit lines, an important limitation in urban areas where multiple transit services spatially overlap.

To address this gap, we introduce two algorithms that go beyond broad mode detection to reconstruct detailed transit itineraries from GPS data. Detailed reconstructions give public transit advocates and transportation planners a far more granular view of transit system usage, capturing, for example, not only that a person took a bus, but exactly which bus line they took, where they boarded and alighted, how long they waited for transfers, and how reliable each connection was. This detailed information enables targeted service improvements (e.g., identifying particular routes or transfer stations that consistently cause delays) and deeper equity analyses (e.g., determining if certain neighborhoods experience longer wait times or more transfers). Our main contributions are:

- GTFS-based spatial indexing: An R-tree index on transit route geometries to efficiently match trajectory segments to specific transit lines with ~80% accuracy.
- Google API-based trajectory matching: An iterative leg-matching algorithm leveraging the Google Maps Directions API with ~85%.
- High-resolution transit metrics: Both methods also extract detailed metrics such as actual transfer wait times, boarding/alighting patterns, and route-specific travel durations.
- Baltimore case study: We evaluate our algorithms on transit trips crowdsourced by Baltimore City residents using the BALTO toolkit [2], which supports community-centered transit advocacy

by surfacing insights on service reliability, transfer efficiency, and equity.

These contributions advance the state of the art in public transit analytics and community-centered advocacy, providing residents, transit advocates, and urban planners with powerful tools for evidence-based transportation planning and service optimization.

2 Related Work

Traditional mobility analysis research has focused on transportation mode detection through manual feature extraction and machine learning classification. Stenneth et al. [6] pioneered this approach by extracting features like heading change rate and velocity and combining GPS trajectories with transportation network data. However, these approaches face limitations including subjective feature selection and sensitivity to varying traffic conditions [5].

Recent work uses deep learning to automatically learn features from raw trajectories, avoiding manual feature extraction. Dabiri and Heaslip [1] showed that convolutional neural networks outperformed traditional methods by as much as 8%. A state-of-the-art attention-based model by Pei et al. [5] achieved 83% accuracy on a public GeoLife dataset (five travel modes). However, when applied to our Baltimore City GPS data, its accuracy dropped to ~53%, consistent with Liang et al.'s findings [3] on cross-city performance degradation. This highlights significant generalization issues due to differences in urban infrastructure, transit systems, and mobility behavior, emphasizing the need for region-specific or tailored approaches.

Reconstructing detailed public transit itineraries remains relatively underexplored. Yazdizadeh et al. [7] proposed an automated approach for trip information extraction but did not address identifying specific transit lines. Our approach builds on these foundations to reconstruct complete transit itineraries with far more detail than previously achieved.

3 Data

3.1 BALTO Dataset

We evaluate our algorithms on crowdsourced GPS trajectories contributed by Baltimore City residents via the BALTO mobile app [2]. Designed for community-centered transit advocacy, BALTO enables residents to contribute their public-transit trips. The dataset comprises 1,607 journeys (2,097 trip legs) from 237 participants collected between September 2023 and October 2024. Participants received incentives to log trips, resulting in a rich collection of daily travel itineraries.

3.2 Data Characteristics

The crowdsourced GPS trajectories are high-resolution, recorded at approximately 2 samples per minute. The data encompasses all major Maryland Transit Administration services in the Baltimore area: local bus, light rail, metro subway, MARC commuter rail, commuter bus, and the Charm City Circulator. About 26% of trips include at least one transfer, reflecting complex multi-modal travel patterns. The median journey duration is 25 minutes, and journeys average 1.3 legs.

3.3 Preprocessing and Preparation

All raw trajectories were preprocessed using the TransBigData library [8] for data cleaning and smoothing. We removed obvious GPS errors or outliers (e.g., implausible speeds >120 km/h or jumps >1000 m between consecutive points), interpolated points to a uniform 10-second interval, and applied a Kalman filter to reduce noise while preserving true movement patterns.

Next, we employed the trackintel toolkit [4] to detect staypoints and segment each trajectory into trip legs. Staypoints were identified using a sliding window: if the user stayed within a 100 m radius for at least 3 minutes, that location was marked as a stop. These parameters capture typical urban transit stop behavior. After identifying stops, we split the trajectory at those points, yielding trip legs, each representing continuous movement between two stops. This segmentation naturally separates walking portions from vehicular transit portions. Figure 1 illustrates the complete preprocessing pipeline, from raw GPS points through cleaning and staypoint detection to final segmented trip legs.

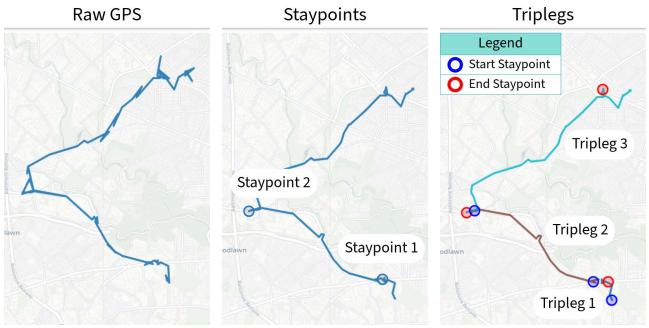


Figure 1: Trajectory processing pipeline.

3.4 Ground Truth Annotation

To create a benchmark for evaluation, we randomly selected 100 journeys (with a total of 232 trip legs) from the dataset for manual annotation. Five researchers independently labeled each trip leg with the ground truth mode and, for transit legs, the specific transit line and direction taken, as well as the boarding and alighting stop locations. Walking segments were also labeled as such. This manual annotation required cross-referencing the GPS traces with transit schedules, Google Maps visualizations, and timestamp information. The resulting annotated set of 232 legs (covering bus, subway, light rail, etc., plus walking connections) provides a robust basis for assessing the accuracy of our itinerary reconstruction algorithms.

4 GTFS-Based Route Matching Algorithm

Our first approach uses General Transit Feed Specification (GTFS) data to automatically label each trip leg with the exact transit line the traveler took, if any. We ingest GTFS feeds from all major transit services in the region and extract key components: route geometries, route identifiers and names, stop locations, and trip schedules. To efficiently query thousands of possible routes, we construct an R-tree spatial index on all transit route polylines, which allows rapid

retrieval of candidate routes by geographic proximity and greatly reduces the search space for matching each trajectory segment.

4.1 Route Matching and Transit Line Identification

For each trip leg produced by preprocessing, the algorithm attempts to identify which transit route matches that segment. We represent the GPS points of the leg as a polyline and perform a spatial query on the R-tree using the segment's bounding box to retrieve candidate routes that overlap the leg's area. We then evaluate each candidate route's similarity to the GPS segment using a two-dimensional Longest Common Subsequence (LCSS) measure. In our implementation, a GPS point and a route shape point are considered a match if they are within 50 m and 60 seconds of each other, reflecting typical GPS accuracy and timestamp jitter. The LCSS algorithm finds the longest sequence of GPS points that align with points along the candidate route. The transit route with the highest LCSS score is selected as the best match for that trip leg. To avoid spurious matches, we also require that the GPS segment covers at least 80% of the candidate route's length (within the spatial tolerance). If no route meets this threshold, the trip leg is left unmatched (no transit line assigned). This conservative criterion favors precision over recall: it may miss some true transit legs if data is noisy, but it correctly leaves walking segments (which have no corresponding transit route) without a false match.

Classical map-matching methods (e.g., FastMapMatching) are optimized for road networks with strong topological constraints. In dense, parallel transit corridors where multiple lines share long, colinear segments, these methods yielded more false positives in our tests by snapping to nearby, visually similar shapes. Our LCSS, combined with the coverage filter, reduced such confusions while remaining robust to GPS jitter and brief detours around stops. We also ran an ablation on our annotated set comparing LCSS, DTW, and Fréchet. LCSS reached ~80% route-matching accuracy. DTW reached ~72% and often overfit trajectories to parallel routes. Fréchet reached ~68% and was sensitive to local noise and small detours near stops.

4.2 Stop Identification and Trip Reconstruction

After a trip leg is matched to a specific transit route, we infer the likely boarding and alighting stops along that route using GTFS stop data. Specifically, we take the stops nearest to the leg's start point and end point as the locations where the user likely boarded and exited the vehicle. In some cases, a single actual transit ride may be split into multiple segments in the GPS data (e.g., due to a lost GPS signal or if our staypoint detection erroneously inserted a break). We handle this by post-processing the sequence of matched legs: if two consecutive legs are matched to the same transit route and the end stop of the first leg is the same as the start stop of the next leg, we merge those legs into one continuous transit ride. This prevents a single real-world trip from being fragmented in the reconstructed itinerary. The final output of the GTFS-based algorithm is, for each journey, an ordered list of legs that includes transit legs (annotated with the identified route and boarding/alighting stops) interleaved with walking segments (for any portions of the journey that did not match a transit route). This reconstructed itinerary provides a comprehensive view of each user's trip, including exactly which transit services were used and where any transfers occurred.

5 Google Maps API-Based Route Matching Algorithm

While the GTFS-based approach is self-contained and incurs no ongoing cost, it relies on the completeness and accuracy of static GTFS data. In complex urban environments, real-world conditions like temporary reroutes, delays, or unofficial stops can cause discrepancies between actual trips and the GTFS-defined routes. To address these gaps, we developed a complementary Google Maps Directions API approach that leverages up-to-date transit information. This method integrates current schedules and real-time data to resolve ambiguities where the GTFS-based approach struggles. In our evaluation it achieved higher accuracy (especially on overlapping routes), serving as a best-case benchmark. The trade-off is reliance on internet access and potential API usage costs and rate limits.

5.1 Query Construction and Temporal Alignment

For each trip leg, we issue a Directions API query in transit mode using the leg's start coordinates as the origin, the end coordinates as the destination, and the leg's start time as the departure time. Because the API cannot retrieve historical schedules, we align each request to a near-future date on the same weekday and time, assuming weekly schedule patterns. We also request multiple route alternatives to maximize the chance of retrieving the actual route taken.

5.2 Multi-level Querying

Many transit journeys involve multiple legs and transfers. We adopt a multi-level querying strategy to capture the full itinerary. First, we issue a full-journey query from the trip's overall origin to its final destination. This provides a complete recommended transit itinerary, including any transfers. Next, we perform segment-level queries for each individual trip leg. Additionally, for each segment we also request driving directions for the same origin and destination. This serves as a control to detect cases where a leg might actually have been by car or rideshare instead of transit; it also helps filter out obviously impractical transit suggestions. In our dataset, participants were dedicated transit riders, but this check adds robustness in case of anomalies.

To assemble the final itinerary, we prioritize the segment-by-segment results and use the full-journey plan as a backup. If individual leg queries miss a connection or yield an inconsistent sequence, we fill gaps using the one-shot full journey plan. When the two sources differ, we choose the route sequence that best fits the GPS data, both spatially and temporally. This ensures a robust reconstruction even if our staypoint-based segmentation was imperfect or the traveler's actual path wasn't the API's top suggestion.

5.3 Route Parsing and Selection

From each returned alternative, we extract its sequence of legs. For transit legs we record the transit line name/number, direction,

Table 1: Itinerary reconstruction performance, 100 journeys.

Algorithm	Precision	Recall	F1-score	Accuracy
GTFS-based method	82.0%	78.0%	80.0%	80.2%
Google API-based method	86.0%	85.0%	85.5%	85.3%

boarding and alighting stops, and scheduled times; for walking legs we capture the distance and duration. We also decode each leg's polyline to obtain the full spatial path.

To identify the best match for the actual GPS trajectory, we compute the Hausdorff distance between the leg's GPS points and each candidate route polyline. The alternative with the smallest distance is selected. This approach is effective even with parallel routes, since the correct line remains closest to the GPS trace while others diverge. We require the chosen route's Hausdorff distance to be under 100 m (~one city block); if no alternative lies within 100 m of the GPS track, the leg remains unmatched.

After selection, the leg is labeled as a transit ride with the identified line and stops; if no transit match is found, it remains a walking segment. The API method naturally distinguishes modes: any segment returned as "walking" stays labeled as walking, and each transit segment is labeled with its specific line. Thus, the trip's complete multi-modal itinerary is obtained directly from the API results combined with our matching logic.

6 Results

We evaluated both algorithms on an annotated set of 100 Baltimore transit journeys (232 trip legs). Table 1 summarizes each method's precision, recall, F1-score, and accuracy. Both performed well, with the API-based approach outperforming the GTFS-based method across all metrics.

As illustrated in Figure 2, both algorithms successfully reconstruct complete transit itineraries by identifying the specific transit line for each leg and the corresponding boarding and alighting stops. These details far exceed what conventional mode detection provides, as the latter yields only broad categories without any route or stop information.

Each approach has distinct trade-offs. The GTFS-based method runs offline on open data at minimal cost. In contrast, the API-based method achieves higher accuracy and is more robust to trajectory quirks, but it relies on an external service and incurs greater overhead. Thus, if offline capability or zero cost is paramount, the GTFS approach is preferable. If maximizing accuracy is critical and external API use is feasible, the API approach is justified despite its dependencies.

However, the API method faces practical limitations. Each Directions API query may incur costs beyond free quotas and is subject to rate limits. In our experiments, processing 232 trip legs stayed within the free tier, but scaling to thousands of trips would require careful budgeting and batching of requests under Google's terms of service. Therefore, despite its strong performance, the API-based method should be applied judiciously in operational settings.

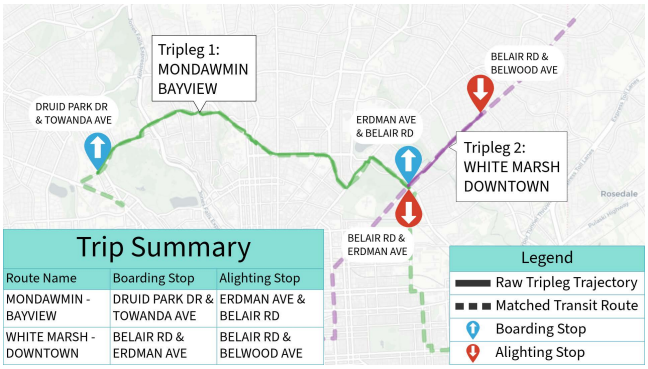


Figure 2: Reconstructed public transit itinerary.

7 Conclusion

We presented two complementary algorithms for reconstructing detailed public transit itineraries from crowdsourced raw GPS trajectories. These methods move beyond coarse mode detection by identifying exact routes and stops, providing a much richer understanding of individual transit usage. The GTFS-based approach demonstrates that effective spatial indexing and sequence alignment can achieve high accuracy in matching GPS data to transit lines using only open data. The Google API-based approach uses an external routing engine to achieve higher accuracy, illustrating the benefits of integrating real-time transit information and advanced routing logic. In future work, we will improve generalization via automatic tuning across cities. Because it only requires GTFS or a standard transit API, it can be applied without major changes. We will also use reconstructed itineraries to supervise models and enable real-time inference for service monitoring. Code and configuration files will be released by January 15, 2026 at https://github.com/ojasjagtap/beyond-mode-detection.

References

[1] Sina Dabiri and Kevin Heaslip. 2018. Inferring transportation modes from GPS trajectories using a convolutional neural network. *Transportation Research Part C: Emerging Technologies* 86 (2018), 360–371. doi:10.1016/j.trc.2017.11.021

[2] Vanessa Frias-Martinez, Saad Abrar, Naman Awasthi, Sunyup Park, and Jessica Vitak. 2023. The BALTO Toolkit - A New Approach to Ethical and Sustainable Data Collection for Equitable Public Transit. In *Proceedings of the 6th ACM SIGCAS/SIGCHI Conference on Computing and Sustainable Societies*. 129–133.

[3] Xiaoyuan Liang, Yuchuan Zhang, Guiling Wang, and Songhua Xu. 2019. A Deep Learning Model for Transportation Mode Detection Based on Smartphone Sensing Data. *IEEE Transactions on Intelligent Transportation Systems* PP (11 2019), 1–13.

[4] Henry Martin, Ye Hong, Nina Wiedemann, Dominik Bucher, and Raubal Martin. 2022. Trackintel: An open-source Python library for human mobility analysis. doi:10.48550/arXiv.2206.03593

[5] Xiaohui Pei, Xianjun Yang, Tao Wang, Zenghui Ding, Yang Xu, Lin Jia, and Yining Sun. 2024. Travel-mode inference based on GPS-trajectory data through multi-scale mixed attention mechanism. *Heliyon* 10, 15 (2024), e35572. doi:10.1016/j.heliyon.2024.e35572

[6] Leon Stenneth, Ouri Wolfson, Philip S. Yu, and Bo Xu. 2011. Transportation mode detection using mobile phones and GIS information. In *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems* (Chicago, Illinois) *(GIS '11)*. Association for Computing Machinery, New York, NY, USA, 54–63. doi:10.1145/2093973.2093982

[7] Ali Yazdizadeh, Zachary Patterson, and Bilal Farooq. 2019. An automated approach from GPS traces to complete trip information. *International Journal of Transportation Science and Technology* 8, 1 (2019), 82–100. doi:10.1016/j.ijtst.2018.08.003

[8] Qing Yu and Jian Yuan. 2022. TransBigData: A Python package for transportation spatio-temporal big data processing, analysis and visualization. *Journal of Open Source Software* 7 (03 2022), 4021. doi:10.21105/joss.04021